

EIR and EIR-transformer (EIR-intro)

Integrative Post-GWAS Methods

Jiyeon Min
Novo Nordisk Foundation Center for Basic
Metabolic Research, University of Copenhagen

UNIVERSITY OF COPENHAGEN

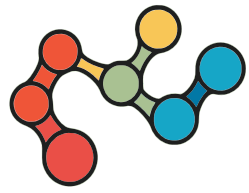


**For this session, you will need the
'eir-intro.ipynb' notebook.**

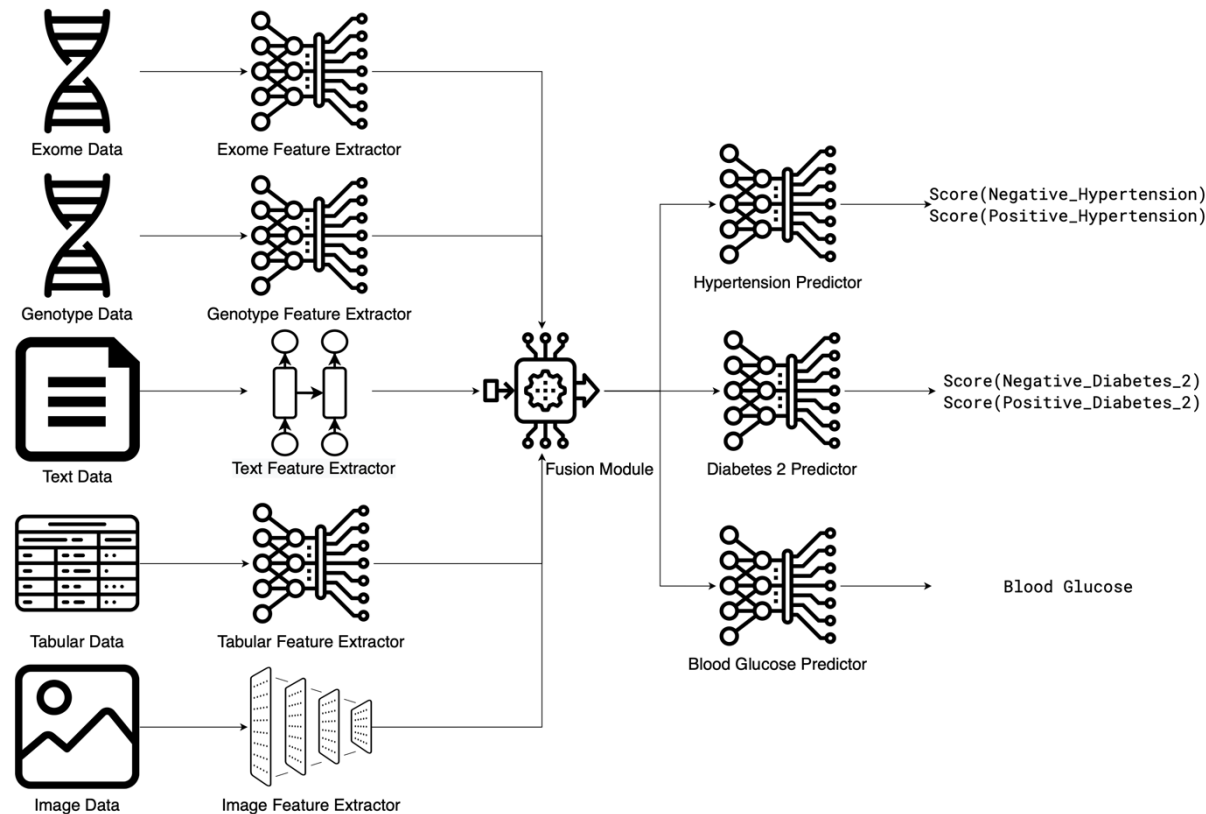
Why this session?

- The next notebook trains a GPT-style transformer on EHR trajectories. It does that by writing a few YAML files and calling ***eirtrain***.
- This session covers what those YAML files are and what those commands do, so the configs are not new when you are also learning about transformers.
- **To avoid learning YAML configuration and transformers at the same time.**

What is EIR?



EIR

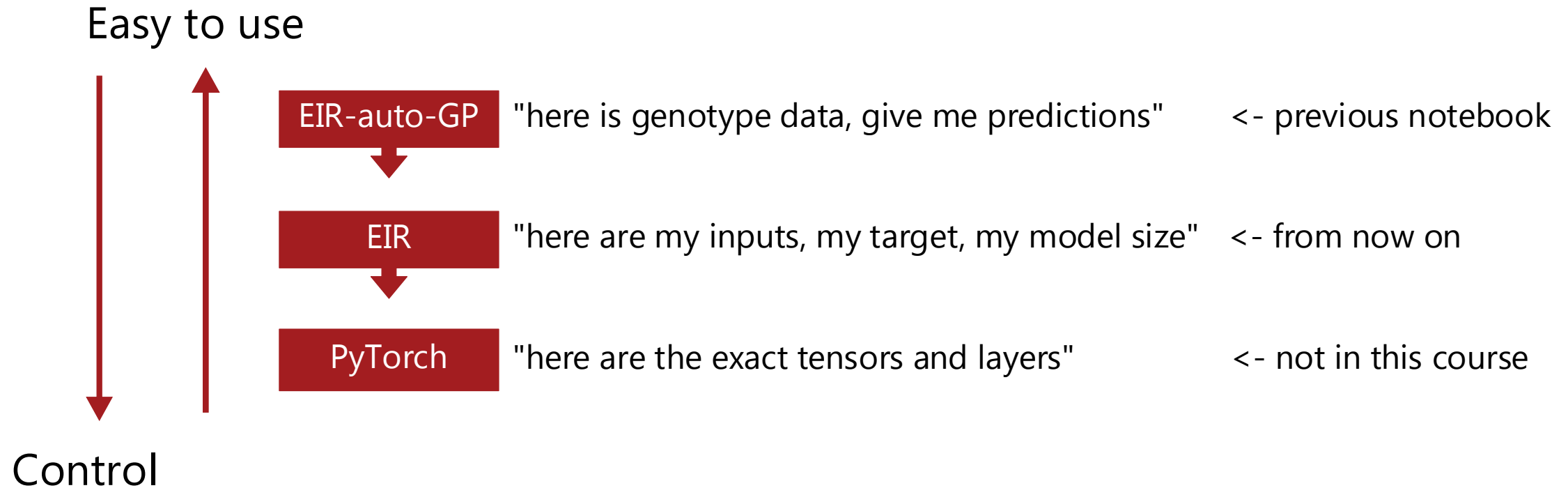


Arnór Ingi Sigurdsson

<https://eir.readthedocs.io/en/stable/>

In short, EIR provides a clean, modular API that connects raw, multi-source inputs to complex downstream tasks, drastically cutting down development and engineering overhead.

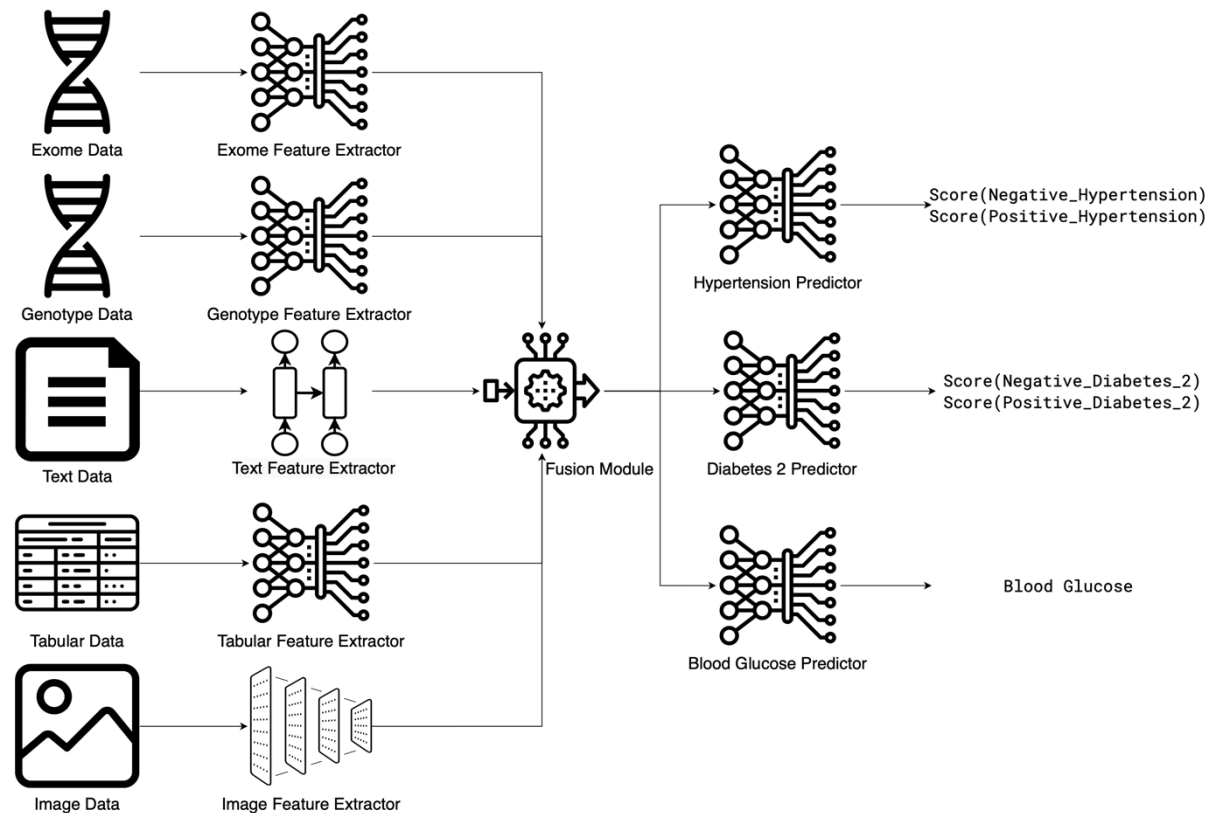
One layer down from EIR-auto-GP



Why YAML instead of Python

- A run is mostly description, very little logic
- You can diff two runs and see the whole difference
- EIR copies the resolved configs into the run folder; Runs are reproducible
- Settings absent from the YAML are at their documented default
- The cost: knowing which file a setting belongs in
 - which is what the next few slides cover!

Every EIR model has the same shape



One config type per step in the diagram
+ the global config for everything that does not belong to a step

The global config

Key	Meaning
memory_dataset: true	load the whole dataset into RAM, which is fine here since the data is small
n_epochs: 20	20 passes over the training data
output_folder	where the run folder goes; everything the run produces lands here
manual_valid_ids_file	sample IDs held out for validation. Fixing them by hand means the plain and GRS models are validated on the same individuals, so losses are comparable
checkpoint_interval: 500	save a checkpoint every 500 iterations
n_saved_models: 1	keep only the best-scoring checkpoint. This is also the default, written out so the behaviour is visible rather than implied
sample_interval: 500	every 500 iterations, generate example sequences so you can see whether training is working
lr: 0.0006	learning rate
latent_sampling	write the internal representation of a named layer to disk. This is how the analysis notebooks get patient embeddings out

The output config

output_info:

output_source: ../reference_data/eir-transformer/df_ehr.csv

output_name: ehr # the name you see everywhere: folders, metrics, layer names

output_type: sequence # "generate a sequence", not "predict a label"

output_type_info:

split_on: " "

max_length: 64

sampling_strategy_if_longer: uniform

min_freq: 10

Tokenization

model_config:

embedding_dim: 64

model_init_config:

num_layers: 4

Model size

Why the plain model has no input config?

- For sequence generation, input and output are the same data
- The model reads a trajectory and predicts its next token
- EIR builds the matching sequence input for you
- So configs/eir-transformer/ has no input_*.yaml at all

When you do need an input config?

```
input_info:  
  input_source: ../reference_data/eir-transformer/df_grs.csv  
  input_name: biomarker_inputs  
  input_type: tabular
```

```
input_type_info:  
  input_con_columns: [...]
```

```
model_config:  
  model_type: tabular
```

The fusion config

```
fusion_type: pass-through
```

- Fusion is where several input branches are combined before the output heads
- The default, **mlp-residual**, flattens each branch and concatenates them
- A sequence representation is 64 positions $\times$ 64 numbers. Flattening it loses which number belonged to which token, and the output module needs that
- **pass-through hands the features on untouched.**

eirtrain: required is not the same as needed

Flag	Required by CLI	In practice
--global_configs	yes	always needed
--output_configs	yes	always needed
--input_configs	no	optional for us
--fusion_configs	no	we need it anyway

Two models: The plain transformer / The model with GRS

The plain transformer

```
eirtrain \  
  --global_configs ../configs/eir-transformer/globals.yaml \  
  --fusion_configs ../configs/eir-transformer/fusion.yaml \  
  --output_configs ../configs/eir-transformer/output_sequence.yaml
```

The plain transformer + **GRS**

```
eirtrain \  
  --global_configs ../configs/eir-transformer-with-grs/globals.yaml \  
  --input_configs ../configs/eir-transformer-with-grs/input_tabular.yaml \  
  --fusion_configs ../configs/eir-transformer-with-grs/fusion.yaml \  
  --output_configs ../configs/eir-transformer-with-grs/output_sequence.yaml
```

What training leaves behind

runs/eir-intro-demo/

├── configs/

│ ├── global_config.yaml

│ ├── input_configs.yaml

│ ├── fusion_config.yaml

│ └── output_configs.yaml

├── **saved_models/** **the checkpoint(s)**

├── serializations/ the vocabulary, so tokens map back to strings

├── **latents/** **whatever latent_sampling asked for**

├── results/ generated sample sequences from each sample_interval

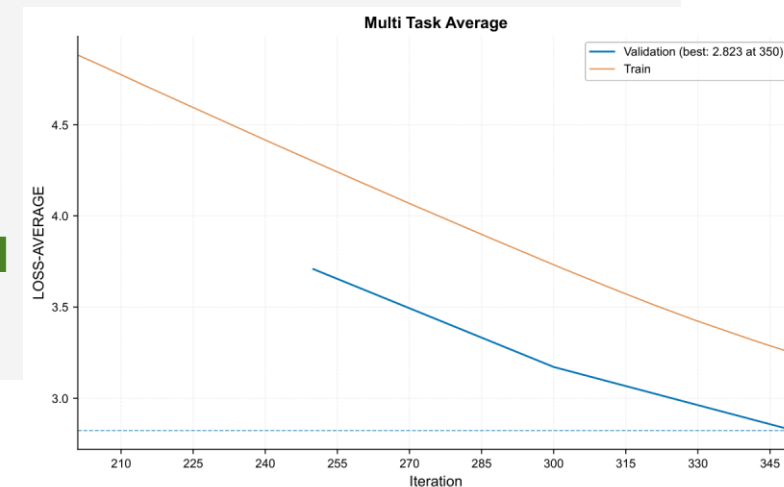
├── meta/

├── **logging_history.log** **the training log**

├── **train_average_history.log** **loss per iteration, as a csv**

├── **validation_average_history.log**

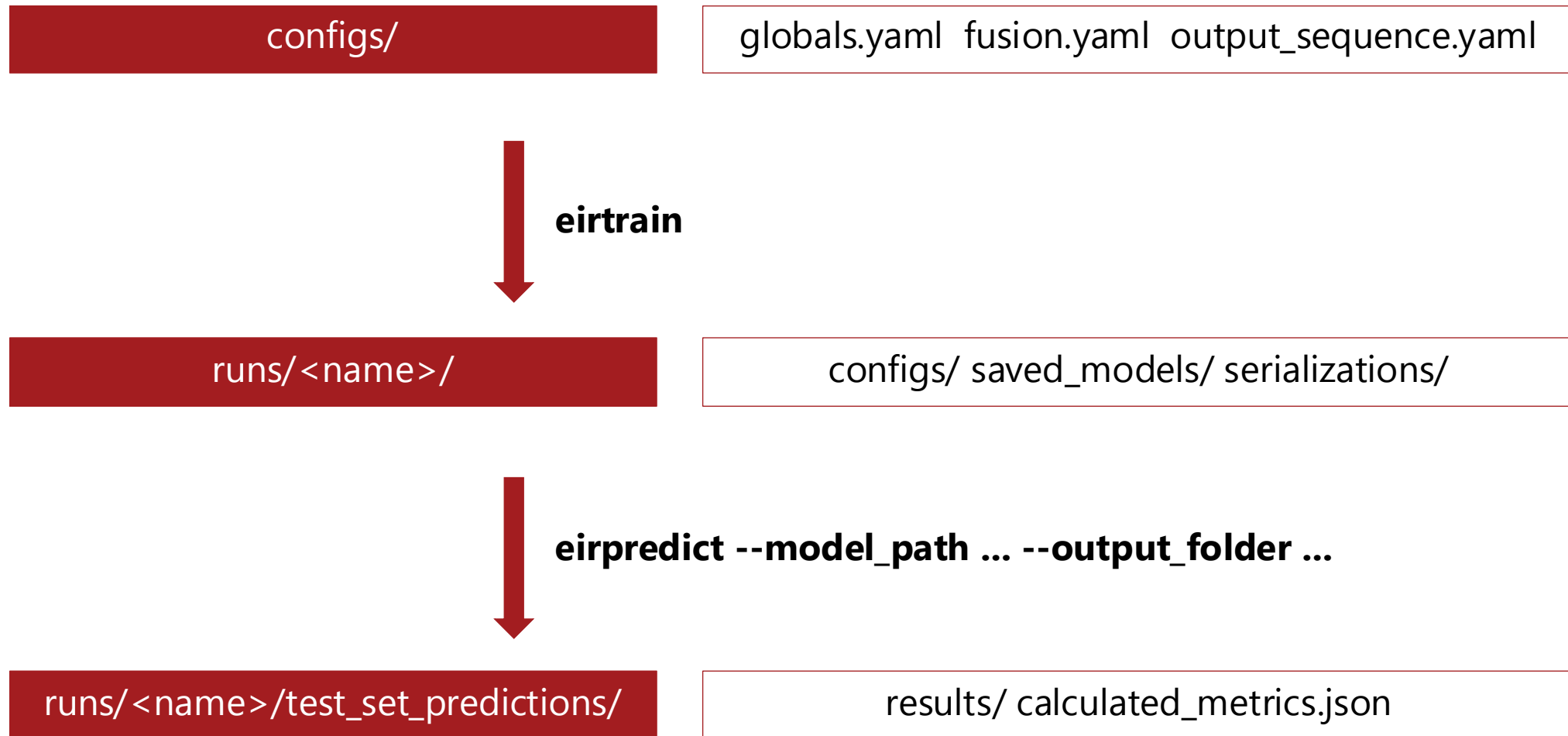
└── **training_curve_LOSS-AVERAGE.pdf** **loss curves, already plotted**



eirpredict

- ***eirtrain*** learns the weights. ***eirpredict*** applies a trained checkpoint to new data.
- `--model_path` and `--output_folder` are the new required flags, as in ***eirautogppredict***.
- The config flags that were required for `eirtrain` are now all optional, including `--fusion_configs`.
- So you only pass the parts that change at prediction time, which is nearly always just the data paths: training csv to test csv.

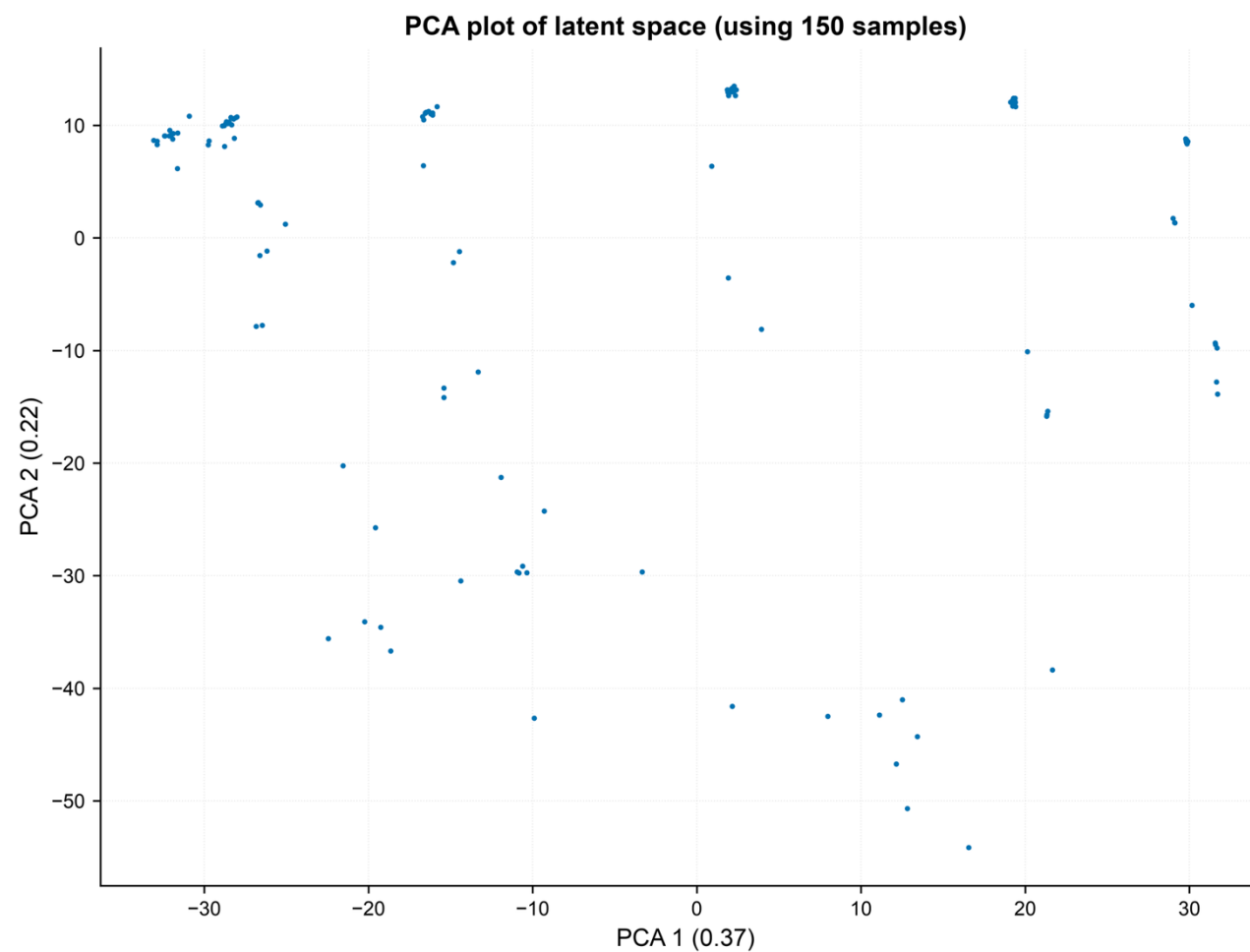
The whole thing on one page



What latent_sampling wrote?

- Every sample_interval, EIR ran the validation set through the model
- It saved the output of output_modules.ehr.final_norm to latents/
- That is the last layer before the model predicts a token
- The array is 150 x 64 x 64
- 150 individuals, 64 positions, 64 numbers per position

PCA of the latent space



!Next!

- eir-transformer.ipynb picks up here
- It simulates EHR data, writes these configs, runs ***eirtrain***
- Then asks whether GRS values as a second input help
- You should now read every YAML there without stopping

Check yourself

- We want 50 epochs instead of 20. Which file changes?
- We want 8 transformer layers instead of 4. Which file changes?
- Our model has one sequence output and no other inputs. Of the two optional flags, which one can we skip, and which one do we still need?
- We asked for `num_layers: 4`, but the model diagram in the next notebook has 8 transformer blocks. Where can we see what was actually built?
- We run `eirtrain` from the repository root instead of from `notebooks/`. What happens?

Answers

- `globals.yaml`, `n_epochs`. Training length applies to the run as a whole.
- `output_sequence.yaml`, `model_config.model_init_config.num_layers`. Model size is set in the config for the output it belongs to.
- We can skip `--input_configs`, since a sequence generation model derives its input from the output config. We still need `--fusion_configs`, because without it we get the default `mlp-residual`, and EIR does not accept that for a sequence-only output.
- In `runs/<name>/configs/` for the resolved configs, where we can also see the input EIR made. There are 4 blocks on the input side and 4 on the output side, which is where the 8 comes from.
- We get Input source `../reference_data/...` does not exist. The paths in the configs are relative, and EIR resolves them from the directory we are running in rather than from the config file, so from the repository root they end up pointing outside it. Absolute paths would avoid this, but then they only work on one machine, so we keep the relative ones and run from notebooks/.



Questions?